COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S1
DATE PERFORMED:	30/07/2026	**DATE SUBMITTED:**	30/07/2026
NAME:	Ramon Brendt D. Campo	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO.4.1 Stack

1. PROCEDURE OUTPUT

MAIN.cpp

```cpp
#include <iostream>
#include "stackList.h"



int main(){

    std:: cout << "---- PUSHING 'C' 'P' 'E' ONTO THE STACK --- \n";


    //push
    push('c');
    push('p');
    push('e');

    //display the current top
    display<char>();
    Top<char>(); //top should be 'e'

    std::cout <<"\n ======== popping all the elements =======\n";


    while(!isEmpty<char>()){

      std::cout<<"Popped: " << pop<char>() <<std::endl;
    }

    std::cout <<"\n ------- Testing the underflow -------- \n";
    pop<char>();

    return 0;


}
```

Stacklist.h

```cpp
#ifndef STACK_LIST_H
#define STACK_LIST_H

//class node that contains data and the next

template <typename T>
class Node{
    public:
    //value that we need to store
    T data; //pointer to the next node
    Node* next;

};

//global pointer for the top of the stack

template <typename T>
Node<T>* head = nullptr;

//operations of the stack

//push
//connect the new node to the current head(the top/pointer)
//its lie the insertinthehead function in linked list

template <typename T>
void push(T newData){
    //creates a new node and assign the newData

    Node<T>* newNode= new Node<T>;
    newNode->data = newData;

    //new node points to the currenyt top, then becomes the new top

    //we assign with head<T> because its a data type
    newNode->next = head<T>;
    //assigns head as the new node (not to null ptr anymore because there is a new npode)
     head<T> = newNode;

}
```

```cpp
//pop
//we will know that the stack is empty if the head pointer i pointed to null meaning there are new nodes
template <typename T>
T pop(){
    //checks if the stack is empty
    if(head<T> == nullptr){
        std::cout << "stack overflow \n";


    }

    //save the current top node
    //creates a temporary holder/variable to be the head pointer/current node
    Node<T>* temp = head<T>;

    //stores the data of the current node
    T tempval = temp->data;
    //move the head to the next node
    //assigns the current pointer of the current node to the next node
    head<T> = head<T>->next;
    //free the removed node but still stores its data
    delete temp;
    //return the removed value
    return tempval;
}
```

```cpp
//top
//returns the top node
template<typename T>
void Top(){
      // error catching:
    if(head<T> == nullptr){

      std::cout << "the stack is empty\n";
      return;
    }

      //display the Top node
      std::cout << "the current top is " << head<T>->data << std::endl;
}


//is_empty
template<typename T>
bool isEmpty(){
      return head <T> == nullptr;

}
```

```cpp
//display all

template<typename T>
void display(){

    if(head<T> == nullptr){

        std::cout << "the stack is empty\n";
        return;
    }

    std::cout << "displaying the stack \n";
    Node<T>* currentNode = head<T> ;


    while(currentNode != nullptr){

      std:: cout << currentNode->data << std:: endl;
      currentNode = currentNode-> next;

    }
    std::cout << "\n";

}

//!note we dont have is_full because the value limit is dynamic
//meaning it doesnt have a fixed value



#endif
```

OUTPUT

```
---- PUSHING 'C' 'P' 'E' ONTO THE STACK ---
displaying the stack
e
p
c

the current top is e

 ======== popping all the elements =======
Popped: e
Popped: p
Popped: c

 ------- Testing the underflow --------
stack overflow
```

ANALYSIS

This program shows how a stack is implemented using linkedlist, its basically a normal linkedlist but it only accesses it through the head, since its the stack it can only enter from the first part or the top of the list. Meaning it wont have the regular components of the normal linked list like the insert head function and compared to an array stack a linked list stack does not have a fixed position mean we can add data until the memory reaches its max capacity. Since this program is missing a display all function I added it as how I will add it to a regular linked list, by setting the current node to the head pointer to indicate its the top and traversed it from there using a while loop.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Stack array

Screenshots

```cpp
#include <iostream>

// attributes of the stack array
const size_t maxCap = 100; // assigns the maximum capacity of the array to 100
char stack[maxCap];        // array to store open symbols
int top = -1;              // shows that the stack is empty

// stack operations
void push(char c);
void pop();
char getTop();
bool isEmpty();
bool isFull();

// helper function for checking balanced symbols
bool isMatchingPair(char open, char close);

int main() {
    // continuous loop to keep program running and prevent terminal syntax errors
    while (true) {
        top = -1; // reset stack top for each new expression
        char ch;
        bool isBalanced = true;

        std::cout << "Expression: ";

        // Reads character by character until the user hits Enter ('\n')
        while (std::cin.get(ch) && ch != '\n') {
            if (!isBalanced) continue; // Skip checking if an error was already found
```

```
31              // If character is an opening symbol, push to stack
32              if (ch == '(' || ch == '{' || ch == '[') {
33                  push(ch);
34              }
35              // If character is a closing symbol
36              else if (ch == ')' || ch == '}' || ch == ']') {
37                  if (isEmpty()) {
38                      isBalanced = false; // missing opening symbol for closing bracket
39                  } else {
40                      char topChar = getTop();
41                      pop();
42                      if (!isMatchingPair(topChar, ch)) {
43                          isBalanced = false; // Error: mismatched bracket types
44                      }
45                  }
46              }
47          }
48
49          // Check if any open symbols were left unclosed
50          if (!isEmpty()) {
51              isBalanced = false; // leftover open brackets in stack
52          }
53
54          // Output Result matching screenshot format
55          if (isBalanced) {
56              std::cout << "Result: VALID (Y)\n" << std::endl;
57          } else {
58              std::cout << "Result: INVALID (N)\n" << std::endl;
59          }
60      }
61
```

```
    return 0;
}

bool isEmpty() {
    return top == -1; // returns true if top is -1 (stack is empty)
}

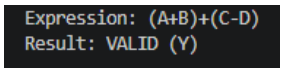
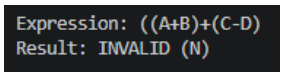
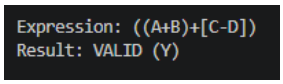
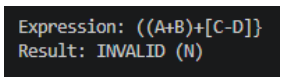
bool isFull() {
    return top >= (int)maxCap - 1; // returns true if top reaches array capacity limit
}

void push(char c) {
    // checks if the stack is full, if it is full it will label stack overflow
    if (isFull()) {
        std::cout << "Stack Overflow!" << std::endl;
        return;
    }
    stack[++top] = c; // increments top index and adds the new symbol
}

void pop() {
    // checks if stack is empty before removing element
    if (isEmpty()) {
        return;
    }
    top--; // decrements top index to remove the top element
}
```

```
char getTop() {
    // if the stack is empty, returns null character '\0'
    if (isEmpty()) {
        return '\0';
    }
    // returns the current top element of the stack
    return stack[top];
}

bool isMatchingPair(char open, char close) {
    // checks if the open and close brackets match each other
    if (open == '(' && close == ')') return true;
    if (open == '{' && close == '}') return true;
    if (open == '[' && close == ']') return true;
    return false;
}
```

Expression	Valid? (Y/N)	Output (Console Screenshot)	Analysis
(A+B)+(C-D)	**Y**		**Since a + b and c - d is enclosed by a parenthesis it will output valid**
((A+B)+(C-D)	**N**		**There is no enclosement onto the first parenthesis meaning its not balanced**
((A+B)+[C-D])	**Y**		**It uses 2 different brackets which is defined in the isMatchingPair function which you can use different brackets as long as its close and open parenthesis**
((A+B]+[C-D]}	**N**		**Since its not the same brackets it wont be recognize by the program as valid therefore its invalid**

SCREENSHOTS

Main.cpp

```cpp
#include <iostream>
#include "delimiter.h"

// Helper function to check matching bracket pairs
bool isMatchingPair(char open, char close) {
    if (open == '(' && close == ')') return true;
    if (open == '{' && close == '}') return true;
    if (open == '[' && close == ']') return true;
    return false;
}

int main() {
    while (true) {
        // Reset stack head for each new expression
        head<char> = nullptr;

        char ch;
        bool isBalanced = true;

        std::cout << "Expression: ";

        // Reads input character by character
        while (std::cin.get(ch) && ch != '\n') {
            if (!isBalanced) continue;

            // If opening symbol, push to stack
            if (ch == '(' || ch == '{' || ch == '[') {
                push<char>(ch);
            }
```

```
30              // If closing symbol, validate against popped element
31              else if (ch == ')' || ch == '}' || ch == ']') {
32                  if (isEmpty<char>()) {
33                      isBalanced = false;
34                  } else {
35                      char topChar = pop<char>();
36                      if (!isMatchingPair(topChar, ch)) {
37                          isBalanced = false;
38                      }
39                  }
40              }
41          }
42
43          // If stack is not empty, unclosed brackets remain -> INVALID
44          if (!isEmpty<char>()) {
45              isBalanced = false;
46          }
47
48          // Output Result
49          if (isBalanced) {
50              std::cout << "Result: VALID (Y)\n" << std::endl;
51          } else {
52              std::cout << "Result: INVALID (N)\n" << std::endl;
53          }
54
55          // Clean up remaining stack memory
56          while (!isEmpty<char>()) {
57              pop<char>();
58          }
59      }
60
61      return 0;
62  }
```

HEADER FILE

```cpp
#ifndef DELIMITER_H
#define DELIMITER_H

#include <iostream>

// class node that contains data and the next pointer
template <typename T>
class Node {
    public:
    T data;      // value that we need to store
    Node* next;  // pointer to the next node
};

// global variable template for the head of the stack
template <typename T>
Node<T>* head = nullptr;

// operations of the stack

// push
template <typename T>
void push(T newData) {
    Node<T>* newNode = new Node<T>;
    newNode->data = newData;
    newNode->next = head<T>;
    head<T> = newNode;
}
```

```cpp
// pop
template <typename T>
T pop() {
    if (head<T> == nullptr) {
        std::cout << "Stack Underflow! \n";
        return T();
    }

    Node<T>* temp = head<T>;
    T tempval = temp->data;
    head<T> = head<T>->next;
    delete temp;
    return tempval;
}

// top
template<typename T>
void Top() {
    if (head<T> == nullptr) {
        std::cout << "The stack is empty\n";
        return;
    }
    std::cout << "The current top is " << head<T>->data << std::endl;
}

// is_empty
template<typename T>
bool isEmpty() {
    return head<T> == nullptr;
}

```

```cpp
// display all
template<typename T>
void display() {
    if (head<T> == nullptr) {
        std::cout << "The stack is empty\n";
        return;
    }

    std::cout << "Displaying the stack: \n";
    Node<T>* currentNode = head<T>;

    while (currentNode != nullptr) {
        std::cout << currentNode->data << std::endl;
        currentNode = currentNode->next;
    }
    std::cout << "\n";
}

#endif
```

Stack list

Expression	Valid? (Y/N)	Output (Console Screenshot)	Analysis
(A+B)+(C-D)	**Y**	`Expression: (A+B)+(C-D)` `Result: VALID (Y)`	**Since a + b and c - d is enclosed by a parenthesis it will output valid**
((A+B)+(C-D)	N	`Expression: ((A+B)+(C-D)` `Result: INVALID (N)`	**There is no enclosement onto the first parenthesis meaning its not balanced**
((A+B)+[C-D])	Y	`Expression: ((A+B)+[C-D])` `Result: VALID (Y)`	**It uses 2 different brackets which is defined in the isMatchingPair function which you can use different brackets as**

			long as its close and open parenthesis
((A+B]+[C-D]}	N	Expression: ((A+B)+[C-D]} Result: INVALID (N)	**Since its not the same brackets it wont be recognize by the program as valid therefore its invalid**

Conclusion/analysis/reflection

I learned the implementation of stacks in both lists and array. From what I've noticed lists for me are harder to make than array but the neat part about lists is that it has no fixed value meaning you can add as much as the memory can run wherein for the array theres a maximum capacity and the maxcap set by the user. The way i structured my supplementary activity I basically just did what I did at ILO B and implemented it into the supplementary, I of course added functions to it to output the brackets and the function and to check if its a valid function or not. Overall compared to linked list I would say this is more understandable to learn but tis still hard, especially figuring out how to make the functions work but with help I am able to finish this activity, it might be late but I can safely say I got the gist of how stacks works

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **GEMINI**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Explanation and giving examples**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used gemini to explain how this function works and I asked gemini to give example for the supplementary**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.



Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.